RV College of Engineering®

Rashtreeya Sikshana Samithi Trust

RV College of Engineering®, Bengaluru.

(An Autonomous Institution Affiliated to VTU, Belagavi)

COMPUTER GRAPHICS

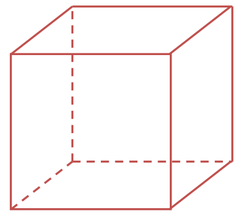
Unit –IV

Visible surface detection: Classification, back-face detection, depth-buffer, scan-line, depth sorting, BSP-tree methods, area sub-division and octree methods

Visible surface detection: Classification, back-face detection, depth-buffer, scan-line, depth sorting, BSP-tree methods, area sub-division and octree methods

Abstract / Overview

- Hidden-surface elimination methods
- Identifying visible parts of a scene from a viewpoint
- Numerous algorithms
  - More memory - storage
  - More processing time – execution time
  - Only for special types of objects - constraints
- Deciding a method for a particular application
  - Complexity of the scene
  - Type of objects
  - Available equipment
  - Static or animated scene



Classification of Visible-Surface Detection Algorithms

- ***Object-space methods*** vs. ***Image-space methods***
  - Object definition directly vs. their projected images
  - Most visible-surface algorithms use image-space methods
  - Object-space can be used effectively in some cases
    - Ex) Line-display algorithms
- Object-space methods
  - Compares objects and parts of objects to each other
- Image-space methods
  - Point by point at each pixel position on the projection plane

Sorting and Coherence Methods

- To improve performance
- ***Sorting***
  - Facilitate depth comparisons
    - Ordering the surfaces according to their distance from the viewplane
- ***Coherence***
  - Take advantage of regularity
    - Epipolar geometry
    - Topological coherence

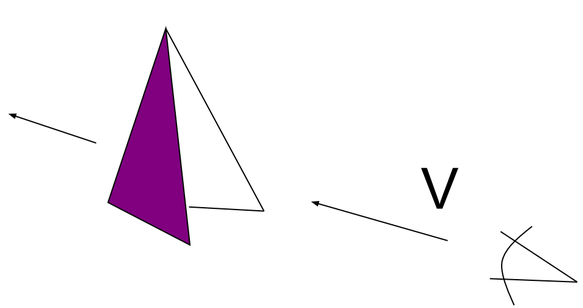
Back-Face Detection - Inside-outside test

- A point (x, y, z) is “inside” a surface with plane parameters A, B, C, and D if

$$Ax + By + Cz + D < 0$$

- The polygon is a back face if

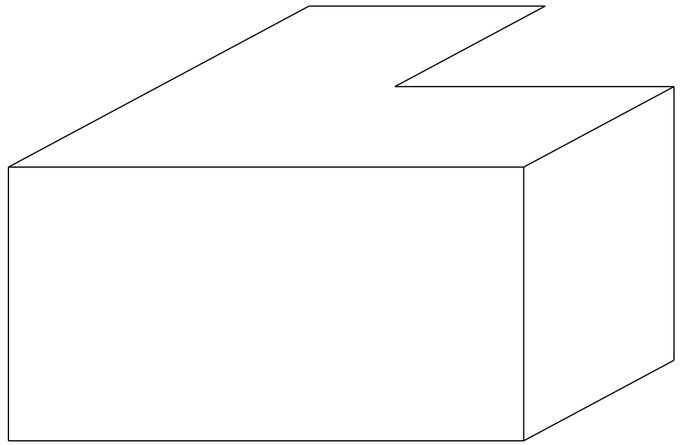
$$V \cdot N > 0$$



  - V is a vector in the viewing direction from the eye(camera)
  - N is the normal vector to a polygon surface

Advanced Configuration

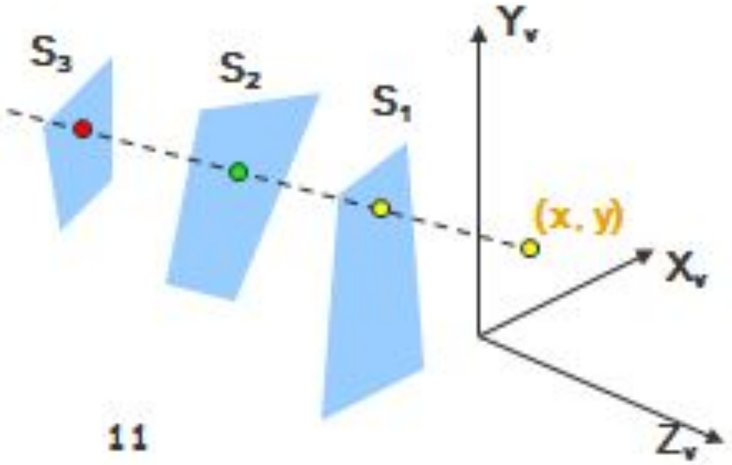
- In the case of ***concave polyhedron***
  - Need more tests
    - Determine faces totally or partly obscured by other faces
  - In general, back-face removal can be expected to eliminate *about half of the surfaces* from further visibility tests



View of a concave polyhedron with one face partially hidden by other surfaces

Depth-Buffer Method Characteristics

- Commonly used image-space approach
- Compares depths of each pixel on the projection plane
  - Referred to as the ***z-buffer*** method
- Usually applied to scenes of polygonal surfaces
  - Depth values can be computed very quickly
  - Easy to implement



Depth Buffer & Refresh Buffer

- Two buffer areas are required
  - Depth buffer
    - Store depth values for each (x, y) position
    - All positions are initialized to minimum depth
      - **Usually 0 – most distant depth from the viewplane**
  - Refresh buffer
    - Stores the intensity values for each position
    - All positions are initialized to the background intensity

Algorithm

- Initialize the depth buffer and refresh buffer

  depth(x, y) = 0, refresh(x, y) = $I_{backgnd}$
- For each position on each polygon surface
  - Calculate the depth for each (x, y) position on the polygon
  - If z > depth(x, y), then set

    depth(x, y) = z, refresh(x, y) = I_{surf}(x, y)
- Advanced
  - With resolution of 1024 by 1024
    - Over a million positions in the depth buffer
  - Process one section of the scene at a time
    - Need a smaller depth buffer
    - The buffer is reused for the next section

Scan-Line Method Characteristics

- Extension of the scan-line algorithm for filling polygon interiors
  - For all polygons intersecting each scan line
    - Processed from left to right
    - Depth calculations for each overlapping surface
    - The intensity of the nearest position is entered into the refresh buffer

Tables for The Various Surfaces

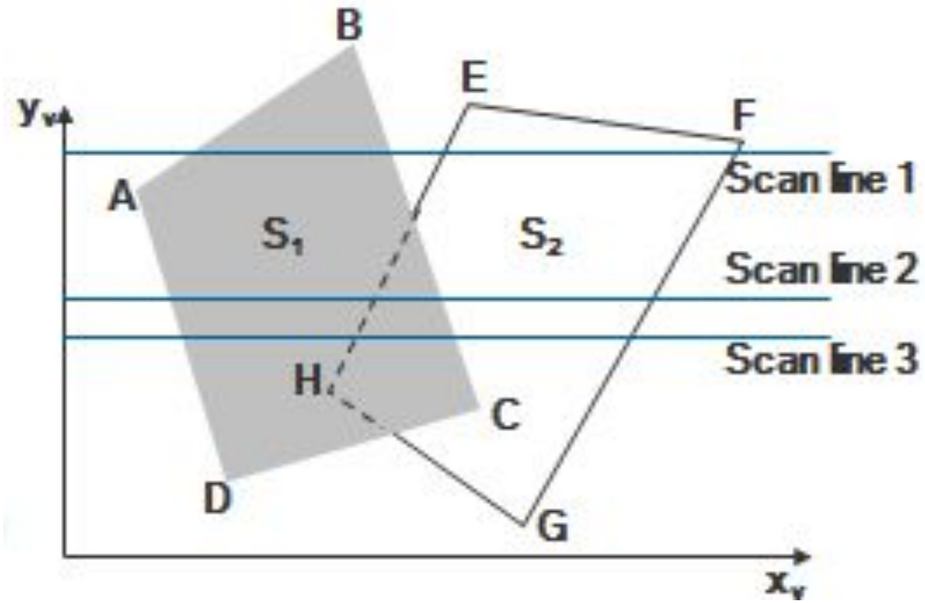
- Edge table
  - Coordinate endpoints for each line
  - Slope of each line
  - Pointers into the polygon table
    - Identify the surfaces bounded by each line
- Polygon table
  - Coefficients of the plane equation for each surface
  - Intensity information for the surfaces
  - Pointers into the edge table

Active List & Flag

- Active list
  - Contain only edges across the current scan line
  - Sorted in order of increasing x
- Flag for each surface
  - Indicate whether inside or outside of the surface
  - At the leftmost boundary of a surface
    - The surface flag is turned on
  - At the rightmost boundary of a surface
    - The surface flag is turned off

Example

- Active list for scan line 1



Edge table

- AB, BC, EH, and FG
- Between AB and BC, only

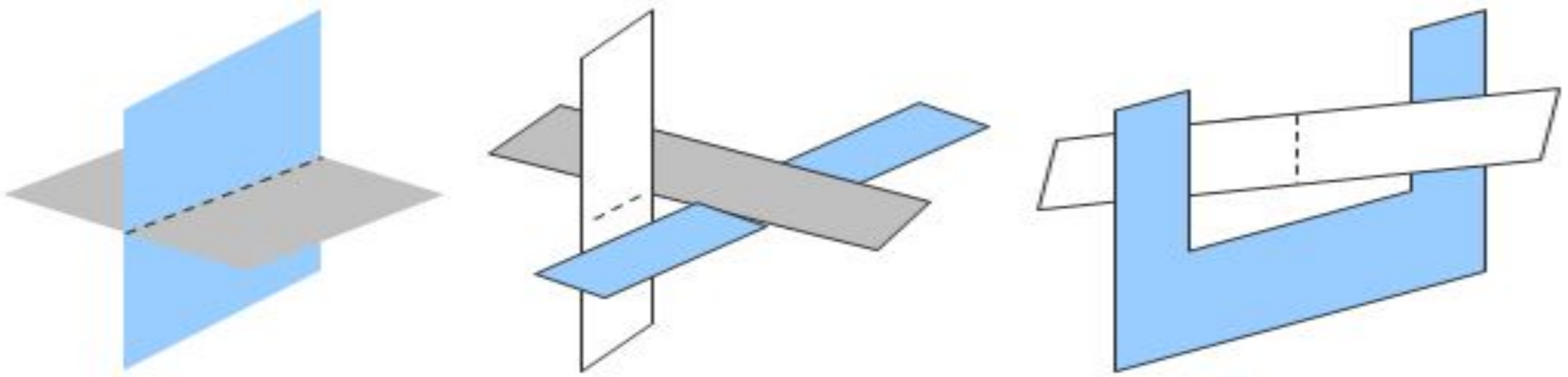
  the flag for surface S_1 is on
  - No depth calculations are necessary
  - Intensity for surface S_1 is entered into the refresh buffer
- Similarly, between EH and FG, only the flag for S_2 is on

Example(cont.)

- For scan line 2, 3
  - AD, EH, BC, and FG
    - Between AD and EH, only the flag for S_1 is on
    - Between EH and BC, the flags for both surfaces are on
      - Depth calculation is needed
      - Intensities for S_1 are loaded into the refresh buffer until BC
  - Take advantage of coherence
    - Pass from one scan line to next
    - Scan line 3 has the same active list as scan line 2
    - Unnecessary to make depth calculations between EH and BC

Drawback

- Only if surfaces don't cut through or otherwise cyclically overlap each other
  - If any kind of cyclic overlap is present
    - Divide the surfaces

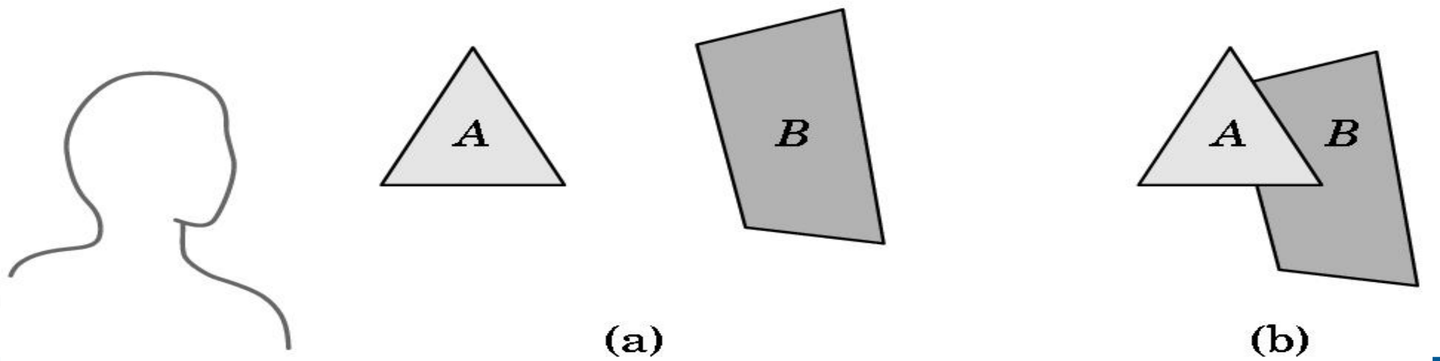


BSP-Tree Method Characteristics

- Binary Space-Partitioning(BSP) Tree
- Determining object visibility by painting surfaces onto the screen from back to front
  - Like the painter's algorithm
- Particularly useful
  - The view reference point changes
  - The objects in a scene are at fixed positions

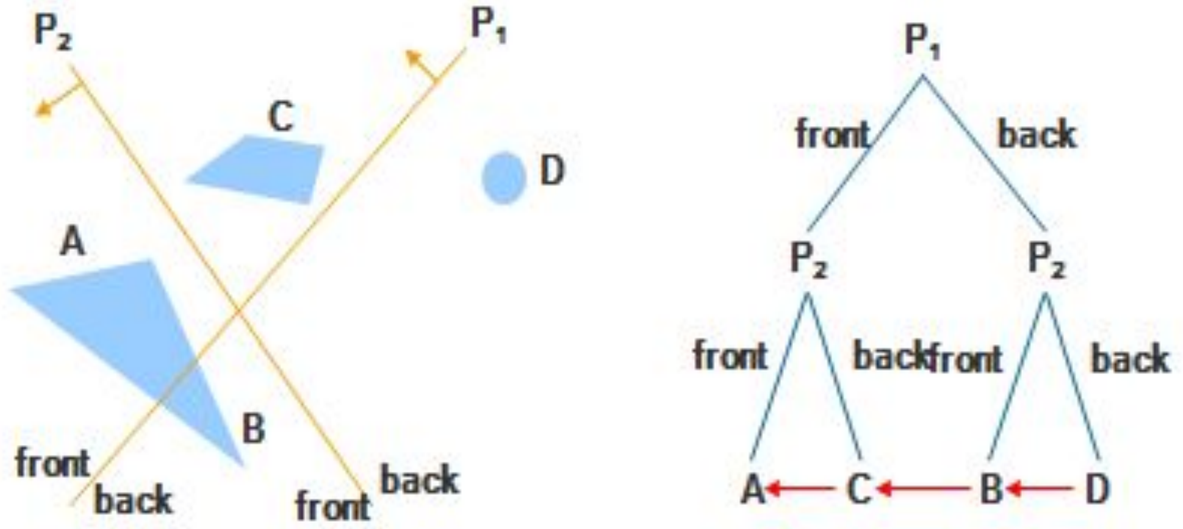
BSP-tree

- The *painter's algorithm* for hidden surface removal works by drawing all faces, from back to front
- How to get a listing of the faces in back-to-front order?
- Put them into a binary tree and traverse the tree (but in what order?)



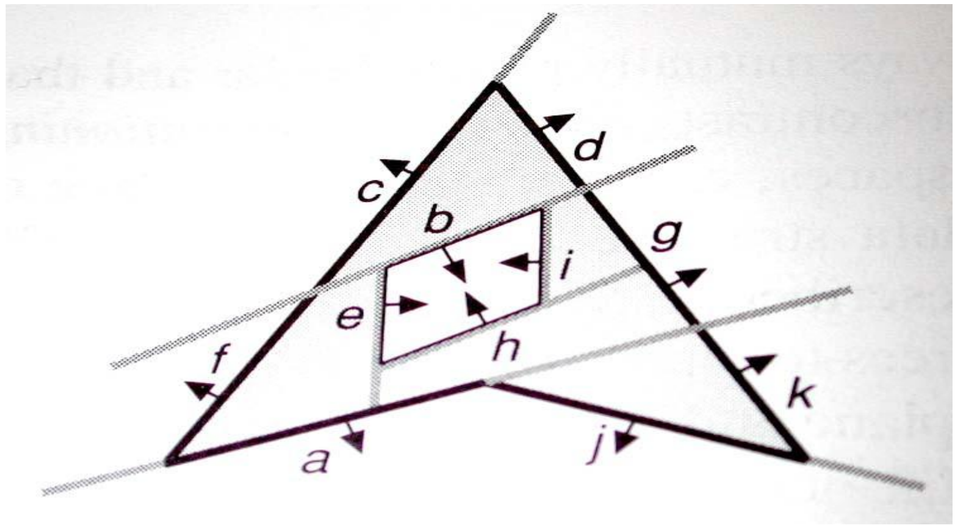
Process

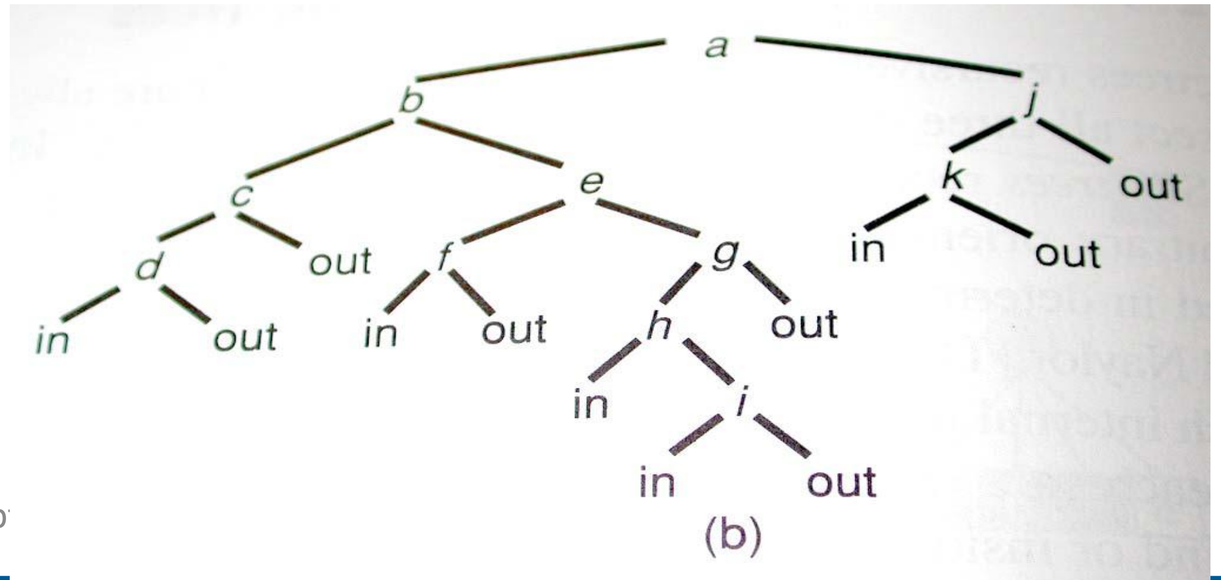
- Identifying surfaces
  - “inside” and “outside” the partitioning plane
- Intersected object
  - Divide the object into two separate objects(A, B)



BSP Tree Figures – example

- Right is “front” of polygon; left is “back”
- In and Out nodes show regions of space inside or outside the object
- (Or, just store split pieces of polygons at leaves)



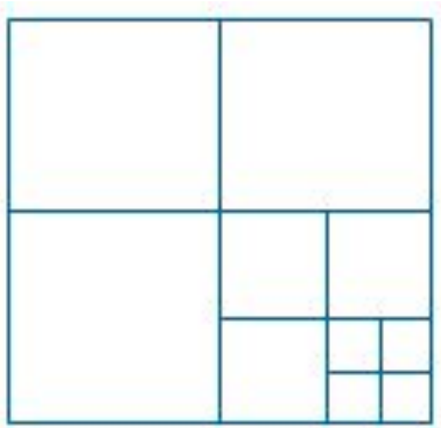


Area-Subdivision Method Characteristics

- Takes advantage of area coherence
  - Locating view areas that represent part of a single surface
  - Successively dividing the total viewing area into smaller rectangles
    - Until each small area is the projection of part of a single visible surface or no surface
  - Require tests
    - Identify the area as part of a single surface
    - Tell us that the area is too complex to analyze easily
- Similar to constructing a ***quadtree***

Process

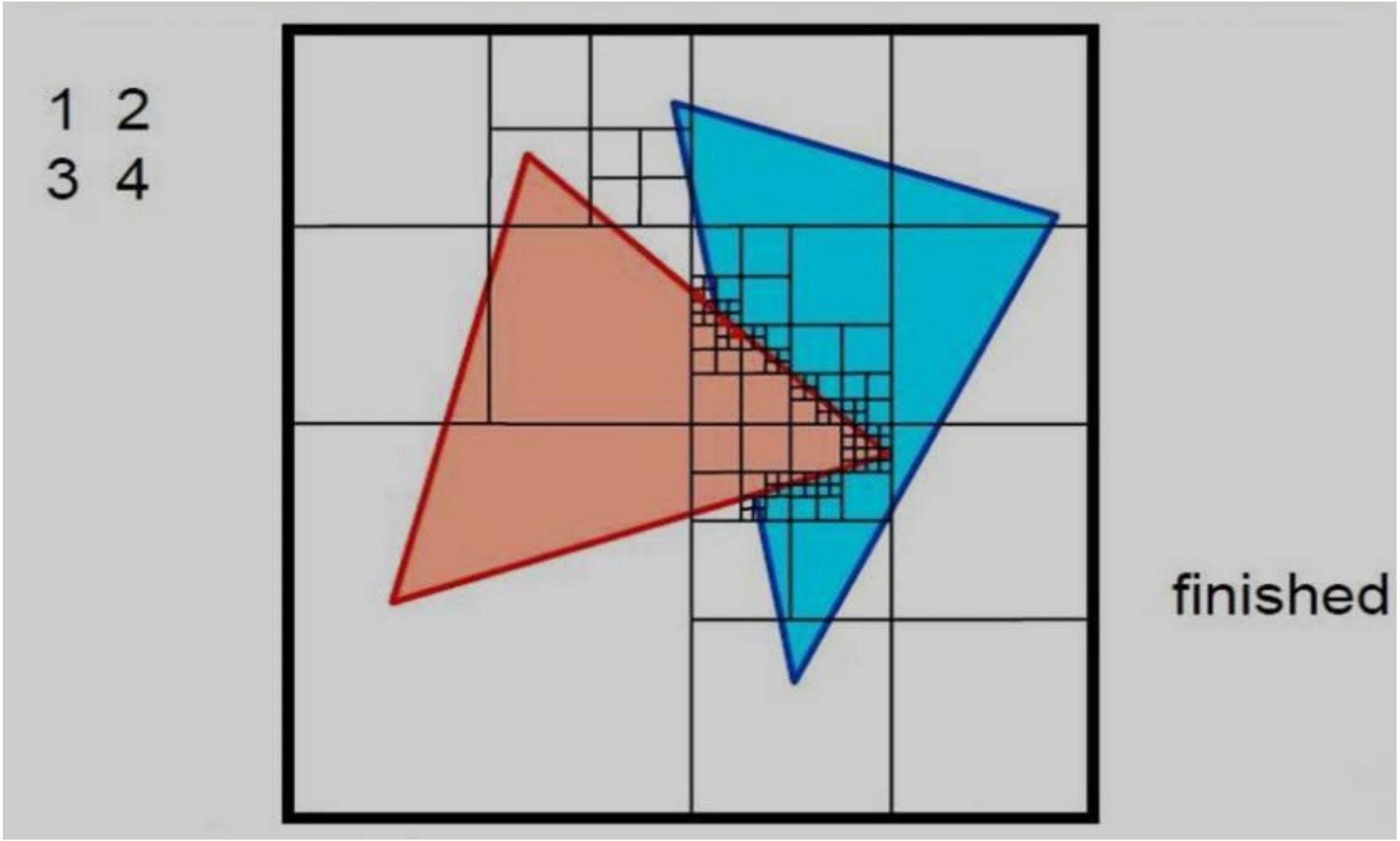
- Staring with the total view
  - Apply the identifying tests
  - If the tests indicate that the view is sufficiently complex
    - Subdivide
  - Apply the tests to each of the smaller areas
    - Until belonging to a single surface
    - Until the size of a single pixel
- Example
  - With a resolution 1024 × 1024
    - 10 times before reduced to a point



Identifying Tests

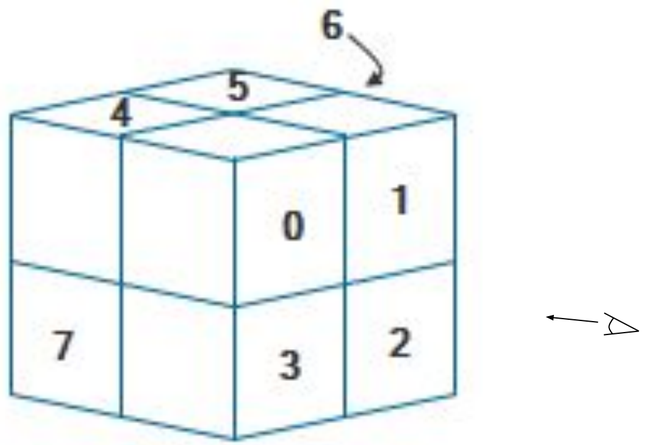
- Four possible relationships
  - Surrounding surface
    - Completely enclose the area
  - Overlapping surface
    - Partly inside and partly outside the area
  - Inside surface
  - Outside surface

- No further subdivisions are needed if one of the following conditions is true
  - All surface are outside surfaces with respect to the area
- Only one inside, overlapping, or surrounding surface is in the area
- A surrounding surface obscures all other surfaces within the area boundaries □ from depth sorting, plane equation


1 2
3 4
finished

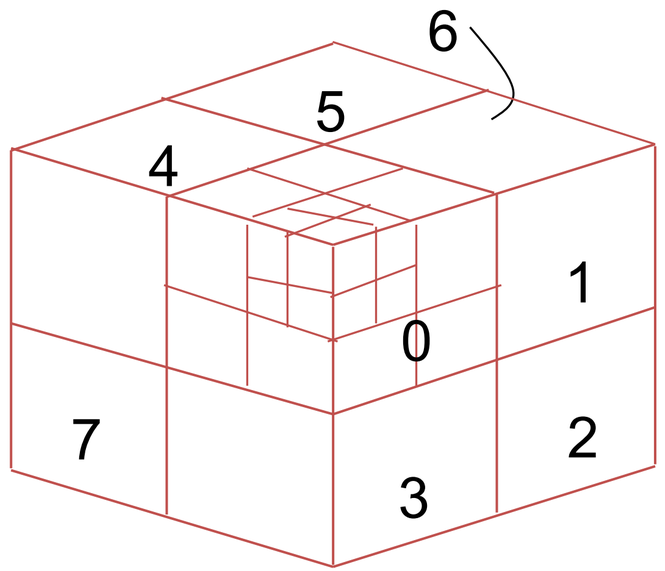
Octree Method - Characteristics

- Extension of ***area-subdivision method***
- Projecting octree nodes onto the viewplane
  - Front-to-back order □□ Depth-first traversal
    - The nodes for the front suboctants of octant 0 are visited before the nodes for the four back suboctants
    - The pixel in the framebuffer is assigned that color if no values have previously been stored
      - Only the front colors are loaded



Displaying An Octree

- Map the octree onto a quadtree of visible areas
  - Traversing octree nodes from front to back in a recursive procedure
  - The quadtree representation for the visible surfaces is loaded into the framebuffer



Octants in Space

Thank You